CREDIT CARD FRAUD DETECTION VIA RANDOM UNDERSAMPLING, SMOTE OVER-SAMPLING, AND PENALIZED LOGISTIC REGRESSION

Author: Aaheri Gangopadhyay

Repository: github.com/aaherigangopadhyaystats-code

ABSTRACT

Detecting credit card fraud is a classic hidden-signal challenge because legitimate transactions far outnumber fraudulent ones. In this project, I analysed 23,323 transaction records to handle severe class imbalance and build a dependable fraud detection model. I explored two different strategies to fix class asymmetry: Random Under-sampling of normal transactions and Synthetic Minority Over-sampling Technique (SMOTE) across the full dataset. A standard Logistic Regression model trained on an under-sampled subset reached 98.27% training accuracy and 96.55% test accuracy. To evaluate performance on a realistic transaction flow without changing the natural fraud rate, I trained an L2-penalized Logistic Regression model using SMOTE-balanced training data. When tested on unseen data (n = 4,665), the model achieved 100.00% Recall for fraudulent transactions—catching every single fraud case while maintaining an overall test accuracy of 99.00%.

1. PROJECT OVERVIEW AND RATIONALE

Financial fraud detection models face a constant real-world trade-off. If a model is too cautious, fraudulent charges go undetected; if it is too sensitive, normal everyday purchases get blocked, creating unnecessary frustration for legitimate cardholders.

I designed this personal project to address three practical modeling questions:

- How severe is class imbalance in real transaction logs, and how much does it distort standard metrics like overall accuracy?
- How do Random Under-sampling and SMOTE synthetic over-sampling compare when trying to highlight rare fraudulent patterns?
- Why are Recall and Precision-Recall AUC far more practical metrics than standard accuracy for financial security pipelines?

2. EXPLORING THE DATASET

I worked with a financial dataset containing 23,323 transaction records across 30 numerical features and a binary target label (Class: 0 = Legitimate, 1 = Fraudulent):

- Time: Elapsed seconds between the current transaction and the first transaction in the dataset.
- V1 to V28: Continuous numeric features anonymized via Principal Component Analysis (PCA) transformation.
- Amount: Transaction amount in local currency.
- Class: Target indicator for transaction status.

Dataset Summary:

Total Records: 23,323 entries (1 incomplete row in V23-V28 and Amount was dropped before final modeling)

Class Distribution: 23,235 Legitimate transactions (99.62%) vs 87 Fraudulent transactions (0.38%)

Missing Values: 1 incomplete row dropped during data cleaning

Exploratory analysis revealed clear differences between normal and fraudulent behavior:

- Transaction Amounts: Legitimate purchases averaged $73.34 (median $17.99), while fraudulent transactions averaged $92.92 (median $1.00). Fraudulent amounts swung wildly, ranging from tiny $0-$1 test authorizations up to a maximum of $1,809.68.
- PCA Feature Distributions: Mean values across PCA features shifted noticeably between classes. For instance, feature V3 averaged +0.78 for normal transactions but dropped to -12.34 for fraud. Feature V4 averaged +0.23 for legitimate transactions versus +6.27 for fraud.

3. PREPROCESSING AND RESAMPLING PIPELINE

To prevent models from blindly guessing "Legitimate" for every transaction (which yields a misleading 99.6% accuracy score), I implemented two resampling techniques:

Approach 1: Random Under-sampling

- Isolated all 87 fraud records.
- Randomly selected an equal number of normal records (n = 492 sampled from the legitimate pool) to build a balanced sub-dataset of 579 total rows.
- Split the balanced subset into an 80% training set (n = 463) and a 20% testing set (n = 116).

Approach 2: SMOTE Over-sampling on Full Partition

- Maintained the natural, imbalanced class distribution for final testing to reflect real-world production conditions.
- Split the cleaned dataset into 80% training (n = 18,657) and 20% testing (n = 4,665).
- Applied SMOTE strictly to the training split, synthesizing artificial fraud examples until training classes reached a 1:1 balance without causing data leakage.

4. MODELING EXPERIMENTS AND FINDINGS

Experiment 1: Logistic Regression on Under-sampled Data

I trained a standard Logistic Regression model on the balanced under-sampled subset (n = 463 training rows).

Training Accuracy: 98.27% (455 / 463 correct predictions)

Test Accuracy: 96.55% (112 / 116 correct predictions)

My Observation: Although under-sampling achieved strong accuracy, discarding 99% of valid transaction data weakens statistical confidence and increases false alarms on unseen normal transactions.

Experiment 2: SMOTE Over-Sampling with Penalized Logistic Regression

To utilize the complete set of legitimate data, I trained an L2-penalized Logistic Regression model (penalty = 'l2', max_iter = 1000) on the SMOTE-balanced training split. I evaluated this model on an unseen test set of 4,665 real transactions (4,648 Legitimate, 17 Fraudulent).

Out-of-Sample Test Set Performance (n = 4,665):

- Overall Test Accuracy: 99.00%
- Legitimate Class (0.0): Precision = 100.00%, Recall = 99.00%, F1-Score = 0.99 (Support = 4,648)
- Fraudulent Class (1.0): Precision = 25.00%, Recall = 100.00%, F1-Score = 0.40 (Support = 17)

Practical Performance Insights:

- Complete Fraud Catch Rate (100.00% Recall): The SMOTE-trained model successfully flagged every single fraudulent transaction (17 out of 17) in the unseen test set.
- Managed Precision Balance: Given the extreme class imbalance (0.38% base rate), reaching 100% Recall meant Precision landed at 25.00%—meaning for every 1 true fraud case caught, roughly 3 legitimate purchases were flagged for manual review. In credit card security, a quick automated verification step for flagged accounts is far better than letting actual fraud slip through.

5. MY TAKEAWAYS AND NEXT STEPS

- Accuracy is Deceptive on Imbalanced Data: A naive model that marks every transaction as non-fraudulent achieves 99.6% accuracy while failing completely. Measuring performance through Recall and Precision-Recall curves is essential.
- Impact of Resampling Strategy: Combining SMOTE over-sampling with L2 regularization allowed the model to learn subtle boundary patterns in PCA space without tossing out large volumes of legitimate transaction data.
- **Future Extensions:** Next, I want to explore non-linear ensemble methods like XGBoost and Isolation Forests, as well as fine-tune decision probability thresholds to balance Precision and Recall for specific business tolerance levels.